

Programación de aplicaciones móviles

1. Selección de entorno de programación	1
2. Herramientas y fases de construcción en Android Studio	2
2.1. Desarrollo del código	2
2.2. Compilación	2
2.3. Ejecución	4
2.4. Depuración	5
2.5. Empaquetado	5
2.6. Firmado y distribución	6
3. Desarrollo del código en Android Studio	6
3.1. XML para el Diseño de Interfaces	6
3.2. Java o Kotlin: Lenguajes de Programación	7
3.3. Herramientas de desarrollo avanzadas	7
3.4. Android Jetpack	7
3.5. Herramientas de Testing	9
4. Interfaz gráfica de usuario (UI) y componentes	10
4.1. Layouts: Estructura de la UI	10
4.2. Widgets: Componentes interactivos	12
4.3. Diseño visual y editor XML	13
4.4. Vista previa en tiempo real	15
4.5. Compatibilidad con pantallas múltiples y densidades	15
5. Gestión de eventos en Android Studio	15
5.1. Listeners en Android: cómo funcionan	16
5.2. Tipos de listeners más comunes en Android	16
5.3. Conexión de listeners en Android Studio	19
5.4. Listeners y ciclo de vida	20
6. Ciclo de vida de las aplicaciones	21
6.1. Ciclo de vida de una Activity	21
6.2. Ciclo de vida de un Fragment	23
6.3. Manejo adecuado del ciclo de vida: Mejores prácticas	25
7. Compilación y ejecución	26
7. 1. Gradle: Sistema de compilación en Android Studio	26
Funciones principales de Gradle:	26
Gradle y el proceso de compilación:	26
7.2. APK vs AAB: Formatos de distribución	27
7.3. Ejecución de la aplicación	27
a) Emulador de Android	27
b) Dispositivo físico	28
7.4. Flujo típico de compilación y ejecución en Android Studio	29
8. Depuración y análisis de rendimiento	29

1. Selección de entorno de programación

A lo largo de este curso, vamos a orientar todo el desarrollo hacia **Android Studio**, debido a que ofrece un entorno completo y avanzado que permite aprovechar al máximo las capacidades de los dispositivos Android, con soporte para los lenguajes más modernos como **Kotlin** y **Java**, y una amplia gama de herramientas para optimizar el rendimiento y la experiencia del usuario.

Además, **Android** es la plataforma móvil más utilizada a nivel mundial, con una cuota de mercado mucho mayor que iOS, lo que ofrece un alcance más amplio para tus aplicaciones. A diferencia de **Java ME**, que ha quedado obsoleto, Android Studio te permite desarrollar aplicaciones modernas, escalables y con una interfaz de usuario rica y adaptable a cualquier dispositivo.

En comparación con **iOS**, Android ofrece mayor flexibilidad y libertad a los desarrolladores, permitiendo una personalización más profunda, un sistema de distribución más accesible, y un hardware mucho más diverso para el que desarrollar.

2. Herramientas y fases de construcción en Android Studio

2.1. Desarrollo del código

En Android Studio, el desarrollo de código se lleva a cabo principalmente en dos lenguajes de programación: **Java** y **Kotlin**.

- **Kotlin**: Es el lenguaje preferido para el desarrollo de aplicaciones Android y recomendado por Google desde 2017. Kotlin es moderno, seguro y conciso, lo que permite escribir código más claro y menos propenso a errores. Gracias a su compatibilidad con **Java** y las herramientas avanzadas que ofrece Android Studio (como autocompletado, refactorización automática y soporte de bibliotecas), Kotlin se ha convertido en el estándar.

Beneficios de Kotlin:

- **Null Safety**: Evita problemas de referencias nulas, que son una de las causas más comunes de fallos en las aplicaciones Android.
- **Coroutines**: Facilita la programación asíncrona y la gestión de operaciones en segundo plano, optimizando el rendimiento sin bloquear la interfaz de usuario.
- **Compatibilidad total con Java**: Puedes usar bibliotecas Java existentes en tus proyectos de Kotlin sin problemas, lo que facilita la migración y reutilización de código.

- **Java:** Aunque Kotlin ha ganado popularidad, **Java** sigue siendo un pilar en el desarrollo de Android, con una base de usuarios muy amplia y una rica colección de bibliotecas. Los desarrolladores con experiencia en Java pueden trabajar fácilmente en proyectos Android, ya que la estructura del código y la funcionalidad son muy parecidas.

2.2. Compilación

Android Studio utiliza **Gradle**, una herramienta de automatización que se encarga del proceso de construcción, administración de dependencias y configuración del proyecto. Gradle permite personalizar la manera en que se compila la aplicación, y es especialmente útil cuando trabajas en proyectos grandes o complejos que requieren diferentes versiones de una misma aplicación.

Gestión de dependencias: Gradle facilita la inclusión de bibliotecas de terceros en tu proyecto. Por ejemplo, puedes agregar rápidamente bibliotecas populares como **Retrofit** para manejo de API o **Room** para bases de datos locales, añadiendo líneas en el archivo `build.gradle`.

```
dependencies {  
  
    implementation "androidx.room:room-runtime:2.4.0"  
  
}
```

- **Variantes de compilación:** Gradle permite manejar diferentes variantes de la aplicación (por ejemplo, versiones **debug** y **release**) dentro de un mismo proyecto. La versión **debug** se usa para pruebas y tiene funciones adicionales como logs y herramientas de desarrollo, mientras que la versión **release** está optimizada para el rendimiento y lista para la distribución.
- **Tareas automatizadas:** Puedes automatizar tareas como la generación de APK, ejecución de pruebas, análisis de código, etc., utilizando scripts personalizados en Gradle.

El archivo `build.gradle` en un proyecto de **Android Studio** se encuentra en dos ubicaciones principales, ya que cada proyecto tiene al menos dos archivos `build.gradle`:

1. `build.gradle` (Project level)

Este archivo está ubicado en el nivel raíz del proyecto. Contiene configuraciones generales del proyecto, como la configuración de Gradle y los repositorios donde Gradle buscará las dependencias. Es aquí donde se suelen definir las versiones globales de las herramientas, los repositorios como Maven o Google, y los plugins como Android Gradle Plugin.

Ubicación: /TuProyecto/build.gradle

2. build.gradle (Module level)

Este archivo está dentro de cada módulo del proyecto, normalmente en el módulo app si estás desarrollando una aplicación estándar. Aquí es donde configuras detalles específicos del módulo, como las dependencias, las versiones de SDK, el minSdkVersion y targetSdkVersion, entre otras configuraciones propias de la aplicación.

Ubicación: /TuProyecto/app/build.gradle

2.3. Ejecución

Android Studio facilita la ejecución de la aplicación durante el desarrollo para realizar pruebas en tiempo real. Puedes ejecutar la aplicación en un **emulador** de Android o en un dispositivo físico conectado por USB.

- **Emuladores de Android:** Android Studio incluye un conjunto de emuladores que permiten simular varios dispositivos Android, desde smartphones hasta tablets. Los emuladores simulan el hardware y el software del dispositivo real, lo que te permite probar diferentes configuraciones sin necesidad de tener físicamente cada dispositivo.

Ventajas del emulador:

- Simulación de diferentes versiones de Android y resoluciones de pantalla.
- Simulación de eventos de hardware como giroscopios, cámaras y redes.
- Herramientas integradas para analizar el uso de CPU, memoria y red.

Desventajas del emulador:

- Consumo de recursos: en equipos con menos capacidad, la ejecución de Android Studio junto con el emulador puede resultar en un rendimiento bajo, ralentizaciones y congelamientos
- Rendimiento no realista: las pruebas de rendimiento que se realizan en el emulador pueden diferir significativamente del comportamiento real en un dispositivo físico, lo que puede llevar a errores de rendimiento inesperados cuando la aplicación se instala en un teléfono real.

- Compatibilidad limitada con hardware: algunas características de hardware (como la cámara o los sensores) no se comportan de manera realista o no están completamente soportadas, lo que limita la capacidad de probar esas funciones en el emulador.
- **Pruebas en dispositivos físicos:** Conectar un dispositivo físico es útil para probar características que dependen del hardware real, como la cámara o sensores. Android Studio reconoce automáticamente el dispositivo una vez que está conectado por USB, lo que permite enviar la aplicación directamente desde el IDE.

2.4. Depuración

La depuración es una parte esencial del desarrollo, y Android Studio ofrece herramientas avanzadas para rastrear errores y optimizar el rendimiento de la aplicación.

Logcat: Es una ventana dentro de Android Studio que muestra los logs del sistema y mensajes generados por la aplicación en tiempo real. Es útil para detectar errores (como excepciones y fallos), además de rastrear el comportamiento general del sistema mientras la app está en ejecución.

Puedes personalizar los mensajes de log en tu código utilizando:

```
Log.d("TAG", "Mensaje de depuración")
```

- **Depurador:** Permite detener la ejecución del código en puntos específicos llamados **breakpoints**, lo que te permite inspeccionar el valor de las variables y el estado del programa en tiempo real. Esto es útil para identificar errores lógicos o de comportamiento que no son evidentes solo mirando el código o el log.
- **Herramientas de rendimiento:** Android Studio incluye un **Profiler** para medir el uso de CPU, memoria y red mientras tu aplicación está en ejecución. Esto ayuda a detectar cuellos de botella, fugas de memoria y operaciones costosas que ralentizan la app.

2.5. Empaquetado

Al finalizar el desarrollo, Android Studio genera los archivos necesarios para distribuir la aplicación. Los dos principales formatos de empaquetado son:

- **APK (Android Package):** Es el archivo ejecutable que contiene la aplicación y todos sus recursos (imágenes, bases de datos, bibliotecas). Este es el formato utilizado para instalar aplicaciones directamente en dispositivos Android.
- **AAB (Android App Bundle):** Introducido por Google como una alternativa más eficiente al APK. Los archivos AAB permiten que Google Play optimice la distribución de la aplicación para cada dispositivo, enviando solo los archivos y recursos necesarios, lo que reduce el tamaño del archivo que los usuarios tienen que descargar. Desde agosto de 2021, Google ha hecho obligatorio el uso de AAB para todas las nuevas aplicaciones que se publiquen en la Play Store, reemplazando al formato APK como el método estándar de distribución de aplicaciones en la tienda.

Android Studio facilita la firma de estos archivos, un paso esencial para distribuir la aplicación en la Google Play Store. El proceso de firma garantiza la integridad de la aplicación y la asocia con un desarrollador verificado.

2.6. Firmado y distribución

Después de generar el archivo APK o AAB, Android Studio ofrece herramientas integradas para:

- **Firma de aplicaciones:** El proceso de firma garantiza la seguridad y propiedad de la aplicación. Es obligatorio para distribuir en Google Play Store. Existen dos tipos de firmas:
 - Firma para pruebas (debug): Android Studio firma automáticamente tus aplicaciones en modo debug durante el desarrollo para que puedas probarlas en dispositivos.
 - Firma para producción (release): Antes de distribuir tu aplicación en la Google Play Store, debes firmarla con una clave privada que asegure que eres el desarrollador autorizado.
- **Distribución:** Puedes integrar tu proyecto directamente con Google Play Console para gestionar el despliegue, seguimiento de usuarios y analíticas.

3. Desarrollo del código en Android Studio

Android Studio es un entorno de desarrollo integrado (IDE) que proporciona todas las herramientas necesarias para desarrollar aplicaciones móviles completas. Facilita el trabajo tanto en la **interfaz gráfica de usuario (UI)** como en la **lógica** de la aplicación. Esto incluye la creación de interfaces de usuario (layouts), el manejo de datos, la integración de servicios de red, y la optimización del rendimiento y el ciclo de vida de las aplicaciones.

3.1. XML para el Diseño de Interfaces

XML (Extensible Markup Language) es el lenguaje utilizado en Android para definir la interfaz de usuario de una aplicación. Con XML, puedes estructurar y diseñar la disposición de los elementos gráficos (como botones, cuadros de texto, listas, etc.) de manera visual o textual. Cada componente de la interfaz gráfica es un elemento dentro del archivo XML que tiene atributos como tamaño, color, posición, márgenes, y comportamiento.

- **Estructura y reutilización:** Los archivos XML en Android se almacenan dentro de la carpeta `res/layout` del proyecto y están vinculados a la lógica del código Java o Kotlin. Este enfoque separa la lógica de la interfaz gráfica, lo que permite un diseño más limpio y fácil de mantener. Las vistas (como `TextView`, `Button`, `ImageView`, etc.) se definen y organizan en los layouts, lo que proporciona una vista jerárquica de la UI.
- **Visual Editor:** Android Studio incluye un editor visual que permite crear las interfaces gráficas arrastrando y soltando elementos. Puedes alternar entre la vista de **diseño visual** y la vista de código XML. Esto es útil tanto para desarrolladores principiantes como para expertos que necesiten ajustes más detallados. La disposición de los elementos puede hacerse mediante layouts como `ConstraintLayout`, `LinearLayout`, y `RelativeLayout`, que permiten organizar los componentes de la UI de diferentes maneras (alineación, márgenes, dimensiones flexibles, etc.).

3.2. Java o Kotlin: Lenguajes de Programación

Java y **Kotlin** son los dos lenguajes de programación utilizados para escribir la lógica de las aplicaciones Android en Android Studio.

- **Java:** Es el lenguaje tradicional para el desarrollo en Android y tiene una base de usuarios muy amplia. A pesar de que **Kotlin** ha sido adoptado oficialmente como el lenguaje principal, **Java** sigue siendo relevante debido a su extensa documentación, ejemplos y bibliotecas compatibles.
- **Kotlin:** Es el lenguaje preferido por Google desde 2017 y ofrece varias ventajas sobre Java. **Kotlin** es más conciso y seguro, lo que permite reducir errores comunes, como las **NullPointerException** (errores de referencia nula). Kotlin también es compatible con el código de Java, lo que permite usar bibliotecas escritas en Java dentro de proyectos Kotlin. Además, incluye funciones más modernas, como **coroutines**, que facilitan la programación asíncrona sin bloquear el hilo principal de la aplicación.

3.3. Herramientas de desarrollo avanzadas

- **Autocompletado de código:** Android Studio ofrece autocompletado inteligente tanto en Java como en Kotlin. Esto facilita la escritura del código al proporcionar sugerencias basadas en el contexto.

- **Refactorización automática:** Cambiar nombres de variables, métodos o clases es sencillo con la opción de refactorización, que actualiza automáticamente todas las referencias en el proyecto.
- **Verificación de errores en tiempo real:** Android Studio muestra errores y advertencias mientras se escribe el código, lo que permite identificar problemas antes de compilar la aplicación. Las herramientas como **Lint** ayudan a detectar problemas de estilo, compatibilidad y posibles errores de rendimiento.

3.4. Android Jetpack

- **Android Jetpack** es una colección de bibliotecas y herramientas que simplifican el desarrollo de aplicaciones Android robustas y escalables. Estas bibliotecas están diseñadas para ayudarte a escribir código más rápido y de mejor calidad, al tiempo que mantienen la aplicación compatible con diferentes versiones de Android.
- **Componentes clave de Android Jetpack:**
 - **LiveData:** Es un componente que permite gestionar y observar datos en la interfaz de usuario de forma eficiente. LiveData se asegura de que la interfaz gráfica esté siempre sincronizada con los datos subyacentes y que los cambios en los datos se reflejen automáticamente en la UI. También respeta el ciclo de vida de los componentes, lo que evita fugas de memoria o actualizaciones innecesarias cuando la actividad o el fragmento ya no están visibles.
 - **Room:** Es una biblioteca que facilita la gestión de bases de datos locales en Android utilizando SQLite. **Room** proporciona una capa de abstracción sobre SQLite, lo que hace que sea más fácil escribir consultas y gestionar la persistencia de datos sin tener que preocuparse por el manejo directo de la base de datos. Room también se integra con LiveData, lo que permite que la UI se actualice automáticamente cuando cambian los datos en la base de datos.
 - **WorkManager:** Facilita la ejecución de tareas en segundo plano que requieren ser ejecutadas de manera confiable, incluso si la aplicación es cerrada o el dispositivo es reiniciado. Es útil para operaciones como la sincronización de datos en segundo plano.
 - **Navigation:** Simplifica la navegación entre actividades y fragmentos dentro de una aplicación. También ofrece soporte para animaciones de transición y navegación basada en gestos entre otros.
 - **ViewModel:** El patrón **ViewModel** ayuda a gestionar los datos de la UI de manera independiente del ciclo de vida de la actividad o fragmento. Esto significa que los datos persisten incluso cuando hay cambios de configuración, como la rotación de pantalla, lo que mejora la eficiencia y la experiencia del usuario. El ViewModel también trabaja de manera

eficiente con LiveData, asegurando que los datos estén siempre actualizados en la UI.

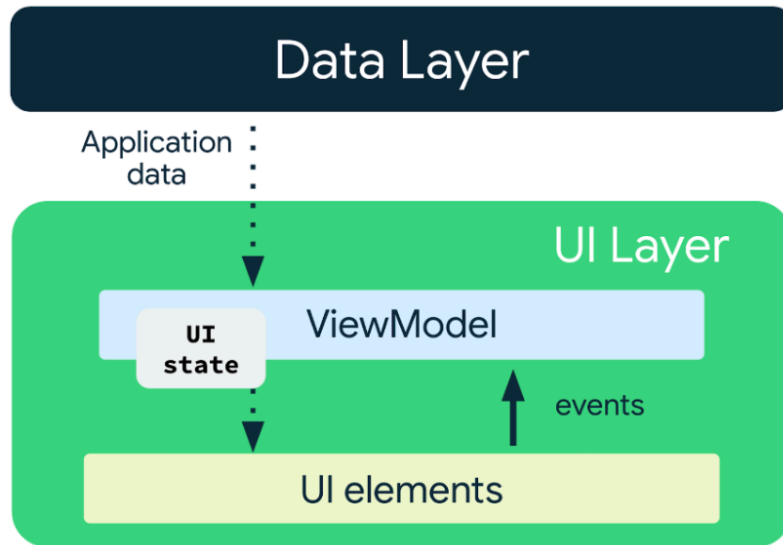


Figura 1: Flujo unidireccional de datos

La lógica en una aplicación puede ser empresarial o de IU:

- La **lógica empresarial** es la implementación de los requisitos del producto para los datos de app. Por ejemplo, agregar un artículo a favoritos en una app de lectura de noticias cuando el usuario presione el botón. Por lo general, esta lógica para guardar un favorito en un archivo o una base de datos se coloca en las capas de dominio o de datos. El contenedor de estado suele delegar esta lógica a esas capas llamando a los métodos que exponen.
- La **lógica de la IU** está relacionada con cómo se muestra el estado de la IU en la pantalla. Por ejemplo, obtener la sugerencia correcta de la barra de búsqueda cuando el usuario selecciona una categoría, desplazarse a un elemento determinado de una lista o establecer la lógica de navegación a una pantalla determinada cuando el usuario hace clic en un botón.

3.5. Herramientas de Testing

Además del desarrollo de la UI y la lógica, Android Studio proporciona herramientas para la **prueba unitaria** y las **pruebas de interfaz**.

- **JUnit:** Es el marco estándar para pruebas unitarias en Android. Con **JUnit**, puedes escribir pruebas para verificar la lógica de tus métodos y asegurarte de que el comportamiento del código sea el esperado.
- **Espresso:** Una herramienta para realizar pruebas automatizadas de la UI. Espresso permite simular interacciones de usuario, como hacer clic en

botones o ingresar texto, y verificar que la interfaz se comporta de manera correcta bajo diferentes condiciones.

4. Interfaz gráfica de usuario (UI) y componentes

Android Studio ofrece un entorno robusto y flexible para diseñar la interfaz gráfica de usuario (UI) de las aplicaciones Android. La UI es una parte crítica del desarrollo de aplicaciones, ya que define cómo los usuarios interactúan con la aplicación. Android Studio permite diseñar esta interfaz utilizando herramientas visuales y código XML, lo que ofrece tanto flexibilidad como facilidad de uso.

4.1. Layouts: Estructura de la UI

Los **Layouts** son los componentes básicos de la interfaz gráfica en Android. Definen la estructura de la disposición de los elementos visuales (widgets) en la pantalla. Android Studio ofrece varios tipos de layouts que puedes usar según tus necesidades:

- **LinearLayout:**

Organiza los elementos en una **fila** (horizontal) o **columna** (vertical). Es sencillo de implementar, pero puede volverse ineficiente si se anida de forma excesiva.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto en un LinearLayout" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Botón" />
</LinearLayout>
```

- **ConstraintLayout:**

Es el layout más flexible y recomendado en la actualidad por Google. Permite posicionar elementos de manera **relativa** entre sí y con respecto a los bordes del contenedor, ofreciendo mayor control sobre el diseño de la interfaz sin necesidad de anidar layouts, lo que optimiza el rendimiento. Es compatible con diferentes tamaños y orientaciones de pantalla, lo que facilita la creación de **interfaces responsives**.

```

<ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Botón"
        app:layout_constraintTop_toBottomOf="@id/textView"
        app:layout_constraintStart_toStartOf="parent" />
</ConstraintLayout>

```

- **RelativeLayout:**

Permite posicionar elementos en relación a otros componentes o al padre del layout. Aunque es más flexible que `LinearLayout`, ha sido en gran medida reemplazado por **ConstraintLayout** debido a su mayor capacidad y eficiencia.

Ejemplo:

```

<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"

```

```
        android:text="Botón"
        android:layout_below="@id/textView"
        android:layout_centerHorizontal="true" />
</RelativeLayout>
```

- **FrameLayout:**

Diseñado para contener un único elemento, aunque puedes agregar más, solo el primero será visible si no especificas reglas. Se utiliza comúnmente para fragmentos o para superponer vistas.

- **GridLayout:**

Organiza los elementos en una cuadrícula de filas y columnas. Es útil para crear interfaces como tableros o interfaces donde se necesita disposición en grillas.

4.2. Widgets: Componentes interactivos

Los **widgets** son los elementos interactivos que componen la interfaz de usuario. Android Studio incluye una gran variedad de widgets que se pueden personalizar para crear una UI funcional y atractiva. A continuación, se describen algunos de los widgets más comunes:

- **Button:** Un botón que los usuarios pueden presionar para realizar una acción. Puede personalizarse con texto, íconos y estilos.

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hacer clic" />
```

- **TextView:** Un componente para mostrar texto. Es uno de los widgets más básicos y se utiliza ampliamente para mostrar etiquetas, mensajes o cualquier tipo de texto en la interfaz.

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Texto de ejemplo" />
```

- **ImageView:** Muestra una imagen en la interfaz. Puede cargar imágenes desde los recursos de la aplicación o desde la web.

```
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/mi_imagen" />
```

- **EditText:** Es un campo de texto editable que permite al usuario ingresar datos.

```
<EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Introduce tu nombre" />
```

- **RecyclerView:** Un widget de alto rendimiento que muestra listas o cuadrículas de datos de manera eficiente, especialmente para grandes volúmenes de datos. Es una versión mejorada de ListView con soporte para animaciones y manipulación de datos en tiempo real.

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

- **Switch:** Un botón de encendido/apagado que permite seleccionar entre dos estados (on/off).

```
<Switch
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Switch" />
```

4.3. Diseño visual y editor XML

Android Studio ofrece dos maneras de diseñar la UI: **Visual Designer** y **código XML**. Ambas herramientas están interconectadas y se pueden usar de manera simultánea para crear interfaces ricas y funcionales.



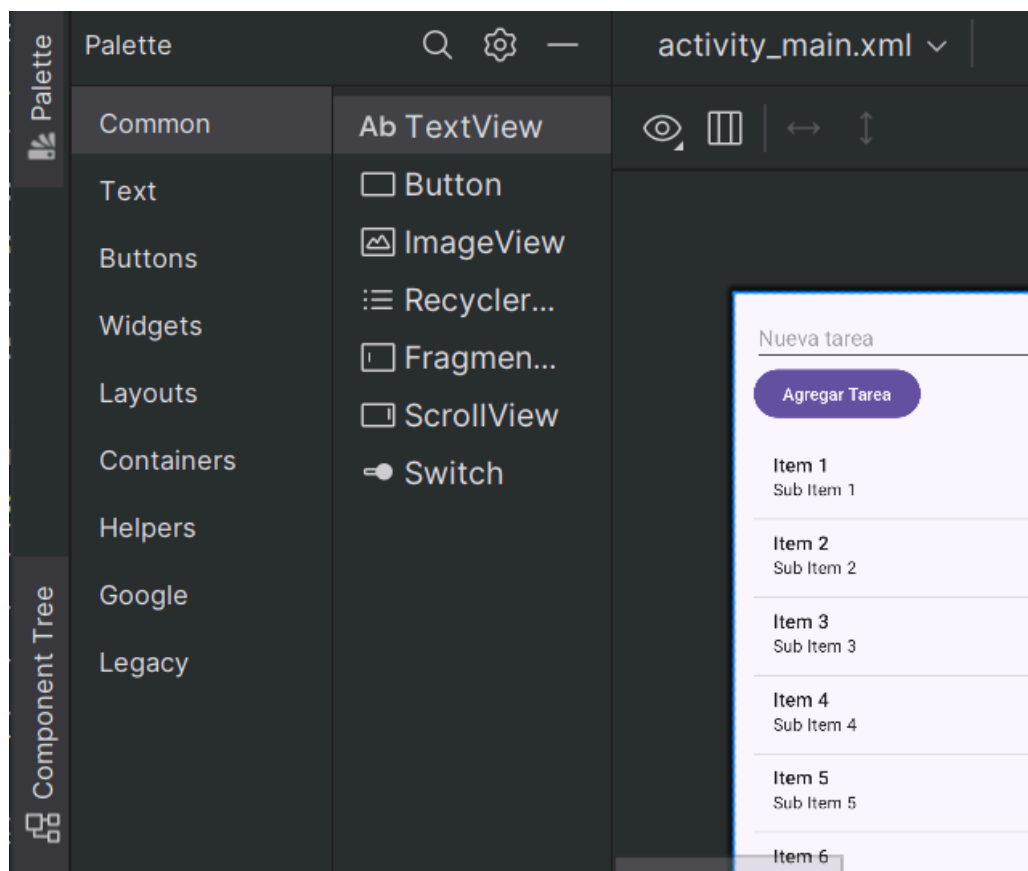
Figura 2. Selector de código XML, mixto o Visual Designer

4.3.1. Editor visual

Android Studio tiene un **editor visual (Layout Editor)** que permite diseñar la interfaz gráfica arrastrando y soltando widgets en el layout. Es especialmente útil para desarrolladores que prefieren trabajar de manera más visual.

Al agregar widgets, el editor genera automáticamente el código XML correspondiente. Puedes cambiar entre la vista de diseño y la vista de código para realizar ajustes precisos si es necesario.

Los elementos del Layout Editor o editor visual son:



1. **Palette:** Una lista de componentes que puedes arrastrar a la pantalla.
2. **Component Tree:** Muestra la jerarquía de los widgets y layouts en el diseño.

3. **Attributes Panel:** Permite ajustar las propiedades de los elementos seleccionados, como tamaños, márgenes, colores, etc.

4.3.2. Vista de código XML

El código XML permite un control más fino y preciso sobre el diseño. Es ideal para hacer ajustes detallados o cuando necesitas un diseño más personalizado y no tan dependiente del editor visual.

Cada elemento de la UI se define mediante etiquetas XML que describen sus atributos, como `layout_width`, `layout_height`, `margin`, `padding`, etc.

Ejemplo de estructura de layouts para diferentes resoluciones:

res/

```
|— layout/           # Diseño base para dispositivos de menor tamaño
|  |— activity_main.xml
|  |— fragment_example.xml
|— layout-sw600dp/    # Diseño específico para dispositivos con ancho >= 600dp
|  |— activity_main.xml
|  |— fragment_example.xml
|— layout-sw720dp/    # Diseño específico para dispositivos con ancho >= 720dp
   |— activity_main.xml
   |— fragment_example.xml
```

4.4. Vista previa en tiempo real

Android Studio permite ver una **vista previa** en tiempo real de cómo se verá la interfaz gráfica en diferentes dispositivos, tamaños de pantalla y orientaciones. Puedes cambiar entre la vista vertical y horizontal o simular dispositivos con diferentes resoluciones para asegurarte de que la interfaz se vea bien en todos ellos.

4.5. Compatibilidad con pantallas múltiples y densidades

Android Studio te permite diseñar una interfaz que sea compatible con diferentes tamaños de pantalla y densidades de píxeles. Para esto, puedes crear diferentes versiones de layouts y recursos gráficos que se ajusten automáticamente a las características del dispositivo.

- **Densidades de pantalla:** Se utilizan diferentes carpetas para proporcionar imágenes con diferentes resoluciones, como `drawable-hdpi`, `drawable-xhdpi`, `drawable-xxhdpi`, etc.
- **Layouts múltiples:** Permite crear varios archivos de layout, uno para cada tipo de pantalla (por ejemplo, para tabletas y teléfonos), en carpetas como `layout-sw600dp` para tabletas.

5. Gestión de eventos en Android Studio

La gestión de eventos en Android Studio permite crear interacciones ricas y dinámicas. Los **listeners** ofrecen un control total sobre cómo la aplicación reacciona a las interacciones del usuario, como hacer clic en botones, tocar la pantalla, realizar gestos o cualquier otra acción y Android Studio facilita su integración tanto en el código como en el diseño de la interfaz gráfica. Esto asegura que la aplicación sea altamente interactiva y ofrezca una excelente experiencia de usuario. Para esto, se utilizan los **listeners**, que son objetos que "escuchan" los eventos y ejecutan acciones cuando estos ocurren. Android Studio proporciona varias formas para gestionar los eventos utilizando Java, permitiendo a los desarrolladores crear aplicaciones interactivas y dinámicas.

5.1. Listeners en Android: cómo funcionan

Un **listener** es un componente que se asocia a un widget (como un botón, un campo de texto, o una vista) y define qué debe ocurrir cuando el usuario interactúa con ese elemento. Por ejemplo, cuando el usuario hace clic en un botón, se puede ejecutar un bloque de código que realiza una acción específica.

En Java y en Kotlin, cada widget puede tener uno o más listeners asociados que gestionan eventos como toques en la pantalla, clics o pulsaciones largas.

5.2. Tipos de listeners más comunes en Android

OnClickListener

Este listener es el más común y detecta cuando el usuario hace clic en un widget, como un botón o un área interactiva. Es ideal para capturar acciones inmediatas.

```
Button button = findViewById(R.id.button);
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Acción a ejecutar cuando se hace clic en el botón
        Toast.makeText(getApplicationContext(), "Botón presionado",
        Toast.LENGTH_SHORT).show();
    }
});
```

El OnClickListener en Android se puede aplicar a cualquier objeto derivado de la clase View, que es la clase base para la mayoría de los elementos de interfaz de usuario en Android. Esto significa que puedes aplicar OnClickListener a casi todos los componentes visuales de una actividad o fragmento, siempre y cuando sean subclases de View.

OnLongClickListener

Este listener detecta una pulsación prolongada en un widget, lo que es útil para mostrar menús contextuales o realizar acciones secundarias.

```
button.setOnLongClickListener(new View.OnLongClickListener() {
    @Override
    public boolean onLongClick(View v) {
        // Acción a ejecutar cuando se mantiene presionado el botón
        Toast.makeText(getApplicationContext(), "Pulsación larga detectada",
        Toast.LENGTH_SHORT).show();
        return true; // Retornar true para indicar que el evento ha sido
        procesado
    }
});
```

OnTouchListener

Este listener se utiliza para detectar toques en la pantalla, movimientos o gestos complejos. Es útil para gestionar interacciones táctiles avanzadas.

```
View view = findViewById(R.id.view);
view.setOnTouchListener(new View.OnTouchListener() {
    @Override
    public boolean onTouch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                // Acción cuando se toca la pantalla
                Toast.makeText(getApplicationContext(), "Pantalla tocada",
                    Toast.LENGTH_SHORT).show();
                break;
            case MotionEvent.ACTION_MOVE:
                // Acción cuando el usuario mueve el dedo en la pantalla
                break;
            case MotionEvent.ACTION_UP:
                // Acción cuando se levanta el dedo
                break;
        }
        return true;
    }
});
```

OnKeyListener

Se utiliza para detectar eventos del teclado, como la pulsación de una tecla. Es útil cuando se desea capturar el ingreso de datos a través del teclado físico o virtual.

```
EditText editText = findViewById(R.id.editText);
editText.setOnKeyListener(new View.OnKeyListener() {
    @Override
    public boolean onKey(View v, int keyCode, KeyEvent event) {
        if (event.getAction() == KeyEvent.ACTION_DOWN && keyCode ==
            KeyEvent.KEYCODE_ENTER) {
            // Acción cuando se presiona la tecla Enter
            Toast.makeText(getApplicationContext(), "Enter presionado",
                Toast.LENGTH_SHORT).show();
            return true;
        }
        return false;
    }
});
```

OnFocusChangeListener

Este listener detecta cuándo un widget, como un campo de texto, gana o pierde el foco. Es útil para validar el contenido cuando el usuario deja de interactuar con el campo.

```
EditText editText = findViewById(R.id.editText);
editText.setOnFocusChangeListener(new View.OnFocusChangeListener() {
    @Override
    public void onFocusChange(View v, boolean hasFocus) {
        if (!hasFocus) {
            // Acción cuando el campo pierde el foco
            validateInput();
        }
    }
});
```

OnDragListener:

Este listener se activa cuando un elemento es arrastrado en la pantalla. Es ideal para gestionar **arrastrar y soltar** elementos en la interfaz.

```
View view = findViewById(R.id.view);
view.setOnDragListener(new View.OnDragListener() {
    @Override
    public boolean onDrag(View v, DragEvent event) {
        switch (event.getAction()) {
            case DragEvent.ACTION_DRAG_STARTED:
                // Acción cuando comienza el arrastre
                break;
            case DragEvent.ACTION_DRAG_ENDED:
                // Acción cuando finaliza el arrastre
                break;
        }
        return true;
    }
});
```

OnScrollChangeListener

Este listener detecta cuando el usuario se desplaza (scroll) dentro de una vista desplazable, como un **ScrollView** o un **RecyclerView**.

```

ScrollView scrollView = findViewById(R.id.scrollView);
scrollView.setOnScrollChangeListener(new View.OnScrollChangeListener() {
    @Override
    public void onScrollChange(View v, int scrollX, int scrollY, int oldScrollX, int oldScrollY)
    {
        // Acción cuando se produce el desplazamiento
        Log.d("Scroll", "scrollX: " + scrollX + ", scrollY: " + scrollY);
    }
});

```

5.3. Conexión de listeners en Android Studio

Android Studio ofrece múltiples formas para conectar listeners a los widgets y gestionar eventos en Java:

1. Asignar listeners en el código:

Como se muestra en los ejemplos anteriores, puedes conectar los listeners directamente en el código Java utilizando métodos como `setOnClickListener()`, `setOnLongClickListener()`, etc. Esta es la forma más flexible y común de gestionar eventos, ya que te permite personalizar la lógica y las acciones que ocurren en respuesta a los eventos.

2. Definir eventos en el archivo XML:

También se pueden definir algunos eventos directamente en el archivo XML del layout. Por ejemplo, para un botón, se puede usar el atributo `android:onClick` para asignar un método que se ejecutará cuando el usuario haga clic en el botón.

```

<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Botón"
    android:onClick="alHacerClick" />

```

El método correspondiente debe ser declarado en la Activity:

```

public void alHacerClick(View view) {
    Toast.makeText(this, "Botón presionado", Toast.LENGTH_SHORT).show();
}

```

Para elementos donde `android:onClick` no está disponible o no es aplicable (como `TextView`, `ImageView`, o contenedores como `LinearLayout`), puedes configurar un `OnClickListener` en el código de la actividad o fragmento.

3. Implementar la interfaz de escucha en la Activity o Fragment:

Otra opción es implementar interfaces de escucha directamente en tu **Activity** o **Fragment**. Esto es útil si tienes varios widgets que necesitan responder a eventos similares, como múltiples botones en la misma pantalla. (Ejemplo: calculadora)

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Asignar el listener a los botones
        Button button = findViewById(R.id.button);
        button.setOnClickListener(this);
    }

    @Override
    public void onClick(View v) {
        switch (v.getId()) {
            case R.id.button:
                Toast.makeText(this, "Botón presionado", Toast.LENGTH_SHORT).show();
                break;
        }
    }
}
```

5.4. Listeners y ciclo de vida

Cuando trabajamos con listeners, es importante tener en cuenta el **ciclo de vida** de la aplicación. Los listeners deben gestionarse adecuadamente para evitar **fugas de memoria** o comportamientos inesperados cuando un componente entra en diferentes estados, como `onPause()` o `onDestroy()`.

Por ejemplo, si tienes un listener que observa eventos de un elemento, debemos asegurarnos de "desregistrar" el listener o detener la escucha cuando ese elemento se detiene o destruye.

6. Ciclo de vida de las aplicaciones

En Android, tanto las **Actividades** (Activities) como los **Fragmentos** (Fragments) tienen un **ciclo de vida** gestionado por el sistema operativo. Este ciclo de vida define cómo una aplicación pasa por diferentes estados, desde su creación hasta su destrucción, y permite que los desarrolladores administren los recursos de manera eficiente. Conocer y manejar correctamente el ciclo de vida de las actividades y fragmentos es esencial para garantizar que la aplicación funcione de manera eficiente, evitar fugas de memoria y optimizar el rendimiento.

6.1. Ciclo de vida de una Activity

Una **Activity** es una pantalla en la aplicación, y su ciclo de vida está controlado por una serie de **callbacks** (métodos) que el sistema operativo Android invoca en respuesta a eventos del sistema o interacciones del usuario, como abrir una nueva aplicación, recibir una llamada o cambiar la orientación del dispositivo.

Las principales etapas del ciclo de vida de una actividad son las siguientes:

1. **onCreate():**

Este es el primer método que se invoca cuando la actividad es creada por primera vez. Es aquí donde se inicializan los componentes de la interfaz de usuario (UI), se establece el layout mediante `setContentView()`, y se configuran otros recursos necesarios.

Ejemplo:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    // Inicialización de componentes de la UI
    Button button = findViewById(R.id.button);
    button.setOnClickListener(v -> {
        // Acción cuando se hace clic
    });
}
```

2. **onStart():**

Después de `onCreate()`, el sistema llama a `onStart()` cuando la actividad se está volviendo visible para el usuario. Aquí es donde la actividad comienza a interactuar

con el usuario, aunque todavía no está en primer plano. La actividad aún no es completamente interactiva en este punto, ya que `onResume()` es el método que permite la interacción completa.

Se puede utilizar para actualizar cualquier recurso visible o preparar la UI antes de que el usuario la vea por completo.

3. `onResume()`

Este método se invoca cuando la actividad entra en **primer plano** y se vuelve interactiva. Aquí es donde se reanuda cualquier operación suspendida cuando la actividad estaba en segundo plano, como reproducción de vídeo, animaciones o la reanudación de tareas que interactúan con el usuario.

Importante: Este es el estado en el que la actividad está completamente visible y lista para recibir interacción del usuario.

4. `onPause()`

El sistema llama a este método cuando la actividad está **perdiendo foco**, por ejemplo, si se abre un cuadro de diálogo o se inicia una nueva actividad sobre la actual. Aunque la actividad sigue visible, ya no está en primer plano, y este es el momento adecuado para pausar operaciones que consumen muchos recursos.

Es importante usar este método para **guardar datos no persistentes** (si es necesario) y liberar recursos que no deben mantenerse activos mientras la actividad está en segundo plano, como animaciones o conexiones a la red.

Ejemplo:

```
@Override
protected void onPause() {
    super.onPause();
    // Pausar tareas intensivas, por ejemplo, detener una animación
}
```

5. `onStop()`

Cuando la actividad ya no es visible para el usuario (porque el usuario navegó fuera de la actividad o se inició otra actividad que cubre toda la pantalla), Android llama a **`onStop()`**. Es aquí donde se deben liberar recursos más pesados que no necesitan mantenerse cuando la actividad está en segundo plano, como el acceso a la base de datos o conexiones de red abiertas.

También es un buen lugar para **guardar el estado de la actividad** si esta puede ser destruida por el sistema para liberar memoria.

6. **onDestroy()**:

Este es el último método que se invoca antes de que la actividad sea destruida. Esto puede suceder porque el usuario cerró la actividad (navegó fuera de ella o presionó el botón de atrás), o porque el sistema necesita liberar memoria y decide destruirla.

Aquí debemos asegurarnos de liberar todos los recursos utilizados, cancelar cualquier tarea pendiente, como por ejemplo hilos y evitar fugas de memoria.

Ejemplo:

```
@Override
protected void onDestroy() {
    super.onDestroy();
    // Liberar recursos finales o cerrar conexiones
}
```

Estado de la actividad en segundo plano

Es importante tener en cuenta que, si bien una actividad puede estar en segundo plano (cuando no está visible pero no ha sido destruida), el sistema operativo Android puede eliminarla en cualquier momento si necesita liberar recursos, como en situaciones de poca memoria. Por eso es esencial guardar el estado de la aplicación y liberar recursos innecesarios cuando una actividad entra en los estados `onPause()` o `onStop()`.

6.2. Ciclo de vida de un Fragment

Los **Fragments** son componentes más modulares y flexibles que permiten gestionar una porción de la UI dentro de una actividad. Los fragmentos tienen su propio ciclo de vida, que está estrechamente vinculado al ciclo de vida de la actividad que los contiene. Esto permite gestionar de manera eficiente cambios en la UI, como el cambio entre pantallas o la reutilización de componentes en diferentes actividades.

El ciclo de vida de un **Fragment** incluye las siguientes etapas clave:

1. **onAttach():**

Es el primer método que se invoca cuando el fragmento se asocia a una actividad.

Aquí puedes realizar configuraciones iniciales relacionadas con la actividad o el contexto que utiliza el fragmento.

2. **onCreate():**

Similar a `onCreate()` en una actividad, aquí es donde se configuran los elementos que deben persistir durante todo el ciclo de vida del fragmento, como **instanciar ViewModels** o manejar configuraciones que no están relacionadas con la interfaz gráfica.

3. **onCreateView():**

Aquí se crea la vista de un Fragment a partir de un archivo XML de diseño (layout). Este proceso se conoce como “inflar el layout” y con él se inicializan los elementos de la interfaz gráfica. Se utiliza para definir qué UI mostrará el fragmento.

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
savedInstanceState) {
    return inflater.inflate(R.layout.fragment_layout, container, false);
}
```

4. **onViewCreated():**

Se invoca justo después de `onCreateView()`, y es el lugar ideal para realizar configuraciones adicionales, como vincular listeners a los widgets o inicializar datos que se muestran en la UI.

5. **onStart() y onResume():**

Los fragmentos siguen el mismo flujo que las actividades. `onStart()` hace que el fragmento sea visible y `onResume()` lo hace interactivo.

6. **onPause() y onStop():**

Estos métodos siguen el mismo principio que en una actividad. Aquí es donde debes liberar o pausar recursos que no deberían mantenerse activos cuando el fragmento ya no está en primer plano o cuando la actividad que lo contiene se pausa.

7. `onDestroyView()`:

Se invoca cuando la vista del fragmento se está destruyendo, lo que sucede, por ejemplo, al cambiar entre fragmentos. Aquí debes liberar referencias a la vista y evitar fugas de memoria.

8. `onDetach()`:

Este es el último método en el ciclo de vida del fragmento, donde se elimina completamente el vínculo entre el fragmento y la actividad. Aquí se libera cualquier recurso que dependa de la actividad.

Interacción entre el ciclo de vida de Activity y Fragment

Los fragmentos están diseñados para interactuar con la actividad que los contiene, y ambos ciclos de vida están interrelacionados. Por ejemplo, cuando una actividad entra en `onPause()`, todos los fragmentos que contiene también pasarán por `onPause()`. Si la actividad es destruida, los fragmentos también lo serán.

Es importante que los desarrolladores gestionen el ciclo de vida del fragmento de manera adecuada, especialmente cuando se manejan transacciones de fragmentos (por ejemplo, reemplazar fragmentos dinámicamente en la UI) para evitar errores o fugas de memoria.

6.3. Manejo adecuado del ciclo de vida: Mejores prácticas

1. **Guardar el estado:** En `onSaveInstanceState()`, guarda el estado de la actividad o fragmento para que se pueda restaurar si el sistema los destruye temporalmente.

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    // Guardar datos en outState
}
```

2. **Liberar recursos a tiempo**, tan pronto como la actividad o fragmento pierdan el foco o se detengan (`onPause()` o `onStop()`).
3. **Evitar fugas de memoria:** Si tienes referencias a **contextos** o vistas en hilos, asegurate de liberarlas en el momento adecuado (por ejemplo, en `onDestroy()` o `onDestroyView()`).

4. **Manejo adecuado de tareas en segundo plano:** Si utilizas tareas en segundo plano como **AsyncTasks**, **LiveData** o **RxJava**, asegúrate de detener o liberar los recursos en los estados adecuados del ciclo de vida.

7. Compilación y ejecución

El proceso de compilación y ejecución de una aplicación Android en Android Studio es manejado principalmente por **Gradle**, un sistema de construcción (build system) flexible que gestiona desde las dependencias hasta la creación del archivo ejecutable final (APK o AAB). Android Studio facilita la construcción, pruebas y distribución de aplicaciones Android mediante un flujo automatizado y configurable.

7. 1. Gradle: Sistema de compilación en Android Studio

Gradle es el sistema de compilación que utiliza Android Studio para gestionar y automatizar todo el proceso de construcción de una aplicación Android. Es una herramienta robusta y flexible que permite definir tareas personalizadas y manejar variantes de compilación de manera sencilla.

Funciones principales de Gradle:

- **Gestión de dependencias:** permite incluir librerías de terceros o módulos externos, como frameworks de desarrollo o SDKs. Estas dependencias se declaran en los archivos de configuración build.gradle (uno a nivel de proyecto y otro a nivel de módulo).
- **Variantes de compilación:** permite configurar diferentes **build types** (tipos de compilación), como **debug** y **release**, que ofrecen configuraciones específicas para pruebas o para producción.
- **Tareas automatizadas:** permite definir y ejecutar tareas automatizadas, como limpieza del proyecto, generación de APKs firmados, ejecución de tests, etc.

Gradle y el proceso de compilación:

El proceso de compilación de Gradle en Android Studio incluye los siguientes pasos:

1. **Sincronización:** Cuando haces un cambio en los archivos build.gradle, Android Studio sincroniza el proyecto con Gradle para asegurarse de que todas las dependencias y configuraciones estén actualizadas.
2. **Compilación del código fuente:** Gradle compila el código Java o Kotlin en archivos de bytes (.class), que luego son convertidos a **Dalvik Executable (DEX)**, el formato de bytecode utilizado por la máquina virtual Android.
3. **Compilación de recursos:** Gradle también procesa los recursos de la aplicación (imágenes, archivos XML, cadenas de texto, etc.), los cuales son empaquetados junto con el código fuente.

4. Generación del archivo ejecutable:

APK (Android Package): El **APK** es el archivo ejecutable que contiene el código compilado, los recursos y el manifiesto de la aplicación. Este archivo puede ser instalado en dispositivos Android o distribuido manualmente.

AAB (Android App Bundle): Un formato más reciente y recomendado por Google para publicar en la Play Store. A diferencia del APK, el **AAB** permite que Google Play genere APKs optimizados para cada dispositivo, reduciendo el tamaño de descarga y mejorando la eficiencia.

Firma de la aplicación: Para distribuir una aplicación, especialmente en la Play Store, el APK o AAB debe estar firmado digitalmente con una clave privada del desarrollador.

7.2. APK vs AAB: Formatos de distribución

APK (Android Package):

- Es el formato estándar de los archivos ejecutables de Android.
- Contiene todo el código, recursos y librerías de la aplicación.
- El APK puede instalarse directamente en dispositivos Android.
- Cuando compilas tu aplicación en modo **debug** para pruebas locales, Android Studio genera un APK.

AAB (Android App Bundle):

- Introducido por Google, es un formato de distribución optimizado.
- Contiene todos los recursos y configuraciones de la aplicación, pero en lugar de generar un APK único, la Play Store genera APKs específicos para cada dispositivo (por ejemplo, con los recursos de idioma o pantalla adecuados).
- Reduce el tamaño de la aplicación que el usuario descarga, optimizando la experiencia.
- Obligatorio para nuevas aplicaciones en la **Google Play Store** desde agosto de 2021.

7.3. Ejecución de la aplicación

Una vez que la aplicación ha sido compilada, Android Studio ofrece dos opciones principales para ejecutarla y realizar pruebas: **emuladores** y **dispositivos físicos**.

a) Emulador de Android

El **Emulador de Android** es una herramienta integrada en Android Studio que simula un dispositivo Android en tu computadora. Permite probar la aplicación sin la necesidad de un dispositivo físico.

Puedes crear múltiples configuraciones de emulador que simulen diferentes dispositivos, con distintas versiones de Android, tamaños de pantalla y especificaciones de hardware.

Ventajas:

Simulación de diferentes tipos de dispositivos y versiones de Android.

Herramientas de depuración integradas, como captura de logs y análisis del rendimiento.

Desventajas:

Consume muchos recursos del sistema, lo que puede hacer que sea lento en computadoras con especificaciones bajas.

Ejemplo de cómo crear un emulador:

1. Ve a **Tools > AVD Manager** (Android Virtual Device Manager).
2. Haz clic en **Create Virtual Device**.
3. Elige un dispositivo (como un teléfono o tableta) y selecciona la imagen del sistema (la versión de Android que deseas emular).
4. Configura el emulador según tus necesidades (tamaño de la pantalla, nivel de API, almacenamiento, etc.).
5. Haz clic en **Finish** y podrás ejecutar tu aplicación en el emulador.

b) Dispositivo físico

Conectar un dispositivo físico a través de **USB** es una de las formas más comunes y precisas de probar una aplicación Android, ya que permite ver el rendimiento real de la aplicación en un teléfono o tableta.

Para habilitar la depuración USB, primero debes activar el **modo desarrollador** en tu dispositivo Android y habilitar la opción de **depuración USB**.

Pasos para habilitar la depuración USB:

1. Ve a **Ajustes > Acerca del teléfono**.
2. Toca el **número de compilación** varias veces hasta activar el **modo desarrollador**.
3. En **Opciones de desarrollador**, habilita la **depuración USB**.
4. Luego, conecta tu dispositivo a la computadora mediante USB y Android Studio debería detectar el dispositivo automáticamente. A partir de ahí, puedes ejecutar tu aplicación directamente en el dispositivo físico.

Ventajas:

Ofrece una prueba más precisa del rendimiento de la aplicación.

Permite probar características que dependen de hardware físico, como la cámara, GPS, sensores, etc.

Desventajas:

Requiere acceso a diferentes dispositivos para probar en distintos tamaños de pantalla y versiones de Android.

7.4. Flujo típico de compilación y ejecución en Android Studio

1. **Escribir el código:** Desarrollas tu aplicación en Java, Kotlin y XML.
2. **Compilación:**
 - Haces clic en el botón Run (o Shift+F10) en Android Studio.
 - Android Studio compila el código, empaqueta los recursos y genera el APK o AAB utilizando Gradle.
3. **Ejecución:**
 - Android Studio instala automáticamente el APK en el dispositivo físico conectado o lo ejecuta en el emulador de Android.
 - La aplicación se inicia en el dispositivo para que puedas probarla.
4. **Depuración:**
 - Android Studio proporciona herramientas de **depuración** como **Logcat**, **Debugger**, **Android Profiler**, y **inspección de memoria** para que puedas ver el rendimiento y los errores mientras la aplicación está en ejecución.

8. Depuración y análisis de rendimiento

Además de compilar y ejecutar la aplicación, Android Studio ofrece herramientas avanzadas para depurar y optimizar el rendimiento de la aplicación:

- **Logcat:** Muestra los mensajes de registro (logs) generados por la aplicación, donde puedes ver errores, advertencias y mensajes de depuración.
- **Android Profiler:** Permite analizar el rendimiento de la aplicación, incluyendo el uso de CPU, memoria, red y batería.
- **Depurador (Debugger):** Te permite colocar **breakpoints** en el código y examinar las variables en tiempo real para rastrear problemas o errores lógicos.

Ejemplo de cómo iniciar el depurador:

1. Establece puntos de interrupción (breakpoints) en el código haciendo clic en el margen izquierdo del editor.
2. Haz clic en el botón de **Debug** en lugar de **Run**.

3. Android Studio ejecutará la aplicación en modo depuración, deteniéndose en los breakpoints y permitiéndote inspeccionar el estado de las variables, la pila de llamadas y el flujo de la aplicación.